

AI in finance

Accounting risk and valuation using Python

David Hayes
2026

Contents

1 Executive summary	2
2 Introduction	2
3 What AI means in a financial context	3
4 Uses in professional finance	3
5 Case Study 1 Tesco and Carillion financial reporting risk	4
6 Case Study 2A LSEG valuation and investment judgement	5
7 Case Study 2B LSEG model governance	5
8 Model risk and professional responsibility	6
9 What the Python analysis establishes	7
10 Conclusion	7
11 Case study appendix	8
12 References	23

The appendix contains Case Study 1 on Tesco and Carillion, Case Study 2A on LSEG valuation and Case Study 2B on governance of the LSEG model.

1 Executive summary

This paper examines accounting risk, valuation and model governance using Python, with AI used as a checking tool. It compares a financial-statement screen applied to Tesco and Carillion, then develops and reviews a valuation of London Stock Exchange Group, or LSEG. The aim is to understand what the calculations reveal and where financial judgement is still needed.

Case Study 1 applies the same accounting rules to Tesco data from 2010 to 2015 and Carillion data from 2010 to 2016. Tesco did not produce a warning specific to the later commercial-income issue, while Carillion produced repeated cash-conversion and working-capital prompts. The comparison exposes both the value and the limits of a simple warning screen.

Case Study 2A uses Python to produce an LSEG discounted cash flow valuation. Case Study 2B examines the governance of that model through calculation checks, source and date controls, sensitivity analysis and review of unresolved assumptions. Together they distinguish calculating a value from deciding how much confidence to place in it.

The main finding is that repeatable calculations need checks on both the evidence and the purpose of the model. Carillion generated useful prompts, Tesco exposed gaps in the screen, and LSEG showed how much a valuation can depend on assumptions.

Methodology and use of AI

The Python analysis was developed for this project. AI was used to check the code and review the analysis. The numerical evidence comes from the scripts, company data and recorded calculation checks. No predictive machine-learning model was trained, and the project did not measure whether AI improved forecasting accuracy.

2 Introduction

2.1 Purpose

Financial analysis requires decisions about which figures are comparable, what an unusual result means and how much confidence to place in a forecast. Automated tools make it useful to distinguish checks that can be performed mechanically from judgements that require understanding of the business.

Tesco and Carillion illustrate why accounting quality matters before a valuation is attempted. The same financial-statement rules can miss a particular accounting issue while revealing weak cash generation elsewhere. LSEG then provides a separate example of how assumptions affect value and how a model should be checked before its conclusions are used.

2.2 Scope and approach

The practical work covers three companies in three case-study sections. Case Study 1 compares Tesco from 2010 to 2015 with Carillion from 2010 to 2016 using the same accounting screen. Case Study 2A values LSEG, and Case Study 2B reviews the governance of that valuation model. Section 8 considers the wider implications for professional responsibility.

The study uses public company information and transparent calculations. Python calculates the Tesco and Carillion ratios and warnings, and generates the LSEG valuation workbook. The accounting screen uses fixed rules. The valuation converts stated assumptions into cash-flow forecasts and tests how changes affect the result.

Sections 5 to 7 summarise the findings. The appendix presents each case under the same headings, with code and outputs alongside the method, interpretation and limitations.

3 What AI means in a financial context

3.1 Machine learning

Machine learning estimates relationships from data rather than relying entirely on rules written in advance. A model might learn patterns associated with loan default or unusual transactions. Its usefulness depends on the quality of the training data and whether those patterns remain relevant when it is used.

A good fit to historical data does not prove that a model will work on new observations. Testing should therefore use data that did not determine the rules. The small Tesco and Carillion datasets are useful for an exploratory comparison, but are insufficient to train and evaluate a predictive model.

3.2 Natural language processing and generative AI

Natural language processing examines written or spoken information. In finance, possible uses include searching annual reports, comparing accounting-policy disclosures and identifying passages that need review. A change in language may provide a useful question without proving that the underlying business has changed.

Generative AI can summarise documents and suggest explanations or code changes. Its responses can sound convincing even when a figure, reference or formula is wrong. Summaries need to be checked against original sources, and calculations need separate tests.

3.3 Automation and decisions

Automation includes tasks such as applying a formula to every row in a dataset. Decision support uses that output to inform a choice. Automated decision-making goes further by allowing a system to act within defined limits. These activities should not be treated as interchangeable.

In the Bank of England and FCA's 2024 survey, 55% of reported AI use cases involved some automated decision-making, while 2% were fully autonomous. These figures describe the survey responses, rather than every financial institution. They show that oversight can take different forms and that it is too broad to say that AI never makes decisions (Bank of England and FCA, 2024).

The distinction is practical in this project. Python applies the valuation formulas, but it does not decide whether a 54% EBITDA margin is credible. A warning for Carillion identifies an unusual pattern; a quiet result for Tesco does not establish that supplier income was recognised correctly.

4 Uses in professional finance

The 2024 Bank of England and FCA survey received 118 responses. Three quarters of respondents said they were using AI. Reported uses included internal processes, fraud detection and cybersecurity. This provides evidence of adoption, although it does not measure the quality of every application or the use of AI across all audit and advisory work (Bank of England and FCA, 2024).

4.1 Audit and accounting analysis

Automated analysis can examine transactions or ratios consistently and direct attention towards unusual items. For example, an analyst can compare the growth of receivables with revenue or examine journals posted close to a reporting date. The underlying data must be complete, and the rule must be relevant to the risk.

Document review can help locate disclosures, but audit conclusions require evidence about the accounting treatment. Case Study 1 compares Tesco's quiet screen with Carillion's recurring

working-capital warnings and weak cash conversion. Both results require examination of the underlying transactions and reporting policies.

4.2 Investment research and valuation

Research involves finding financial information, checking definitions and comparing businesses. AI can assist with extraction and explanations, while Python can automate calculations and produce repeatable workbooks. This can make assumptions easier to inspect and change.

Case Study 2A makes the assumptions visible through scenarios and sensitivities. Case Study 2B examines the checks needed when a saved result disagrees with a live formula or a sensitivity calculation mixes assumptions. The apparent precision of a spreadsheet is not a reason to skip review.

4.3 Risk monitoring and reporting

A monitoring system can flag a change in exposure or summarise events for review. Its usefulness depends on the consequences of missing a problem and the cost of investigating warnings that turn out to be harmless. A system producing many alerts may create additional work without improving decisions.

Reports need to connect a result to its financial meaning and identify unresolved issues. Case Study 2B applies this to LSEG by separating calculation checks from source quality and the judgement needed to accept a forecast assumption.

5 Case Study 1 Tesco and Carillion financial reporting risk

5.1 Question and approach

Case Study 1 asks whether the same annual accounting screen gives useful prompts for Tesco and Carillion. The fixed rules compare receivables and inventory growth with revenue growth, and examine profit after tax relative to operating cash flow. Tesco covers 2010 to 2015 and Carillion 2010 to 2016.

Python uses pandas to calculate annual changes, accrual ratios and warning rules for both companies. The scripts export CSV results and use matplotlib to draw the charts. The appendix includes the calculation listings, both companies' results and the separate later review of Tesco's financial deterioration.

5.2 Findings

For Tesco, the working-capital rules produced warnings in 2011 and 2012, but not in 2014 or 2015. They did not isolate the commercial-income issue. Carillion produced receivables warnings from 2011 to 2013 and negative operating cash flow despite positive profit in 2012 and 2013. In 2016, receivables grew 31.0% and inventory 22.6%, against revenue growth of 11.2%, triggering both working-capital warnings.

Carillion's 2016 cash conversion was about 0.57 times profit after tax. The accrual-history warning did not trigger in any eligible year, so the ratio trends and working-capital flags provide different evidence. A separate later Tesco test identified the 2015 loss and weaker financial position; it is reported separately because its rules were introduced after reviewing the original result.

5.3 Implications

The comparison shows that a screen must be assessed against the risk it is meant to examine. Tesco's aggregated figures did not isolate supplier-income recognition. Carillion's repeated profit-to-cash gaps and working-capital movements would have justified closer examination of contracts and cash collection. They do not establish a prediction of collapse.

The useful lesson is to retain both the warnings and the quiet results. The datasets use mixed reporting versions and contain no healthy peer sample, so the comparison cannot establish a detection rate. Adjusting the rules until a known event is detected would weaken the test.

6 Case Study 2A LSEG valuation and investment judgement

6.1 Question and approach

The LSEG project examines whether a Python valuation model can support an investment assessment. It uses FY2025 financial information, a five-year DCF and a comparable-company screen. Case Study 2A sets out the valuation, while Case Study 2B reviews its calculation controls, evidence and remaining model risks.

The Python builder uses a `dcf_case` function for forecast cash flows, discounting and terminal value. It runs the downside, base and upside assumptions and recalculates each cell of the two sensitivity grids. `XlsxWriter` generates the workbook with live formulas and saved results. The appendix shows the function, intermediate cash flows and both complete sensitivity tables.

6.2 Findings

The downside, base and upside DCF values were £95.41, £122.92 and £150.91 per share. The base result was 51.3% above the £81.266 programme-price proxy. These figures are conditional valuation estimates. They do not establish that the market price was wrong or that an investor would earn that return.

About 80.1% of base enterprise value came from discounted terminal value. Changing the 2030 EBITDA margin from 54% to 52.7% reduced the indication to £119.21 per share. The long-term operating assumptions therefore deserve at least as much attention as the arithmetic.

6.3 Implications

The calculation review is set out in Case Study 2B. Peer evidence and the adjustments from enterprise to equity value remain incomplete, so the valuation supports further investigation rather than a purchase recommendation.

Reproducibility allows a reader to follow assumptions into the answer. It does not verify future cash flows.

7 Case Study 2B LSEG model governance

7.1 Question and approach

Case Study 2B asks what checks and controls are needed before relying on the LSEG valuation in Case Study 2A. It uses the existing model and review records to examine calculation accuracy, source dates, assumptions and limits on use. It does not construct another valuation.

Python provides a separate calculation of the three scenarios and all 50 sensitivity cells. The review also considers the earlier spreadsheet-engine recalculation, scenario-switch checks and enterprise-to-equity reconciliation. These checks have different purposes and are reported separately in the appendix.

7.2 Findings

The recorded review reconciled 277 formula results after recalculation in a separate spreadsheet engine. A separate Python calculation matched the three DCF scenarios and all 50 sensitivity outputs. Earlier corrections addressed inconsistent saved results, sensitivity logic and the scenario check. This establishes consistency for the tested calculations.

Other risks remain. About 80.1% of base enterprise value depends on terminal value, some peer inputs are approximate, and the ownership bridge is incomplete. The market-price and share-count proxies were published after the valuation reference date. These issues affect how the result can be interpreted even when the arithmetic reconciles.

7.3 Implications

A useful governance process records sources and changes, separates reported figures from assumptions and checks important calculations independently. Before using the model for an investment decision, the outstanding ownership adjustments and peer inputs would need review, followed by a fresh calculation check.

The case supports the model as a reproducible analytical exercise with stated limitations. It does not claim that an investment committee has approved it or that the checks establish the reliability of its forecasts. Section 8 relates these findings to wider professional responsibilities.

8 Model risk and professional responsibility

8.1 Calculation errors and assumptions

Model risk includes incorrect formulas, unsuitable data and assumptions that do not fit the decision. These risks need different checks. Recalculating a valuation can reveal a formula error, while assessing a margin forecast requires understanding the business. A correct answer to the wrong question can be just as misleading as a calculation error.

The project provides examples of each problem. The LSEG discount-rate mismatch was a calculation issue. Tesco's quiet screen exposed a gap between the available data and the accounting risk. Carillion's warnings required investigation of their cause. The LSEG terminal margin remained a judgement after the formulas were corrected.

8.2 Evidence and oversight

A useful model record should identify the source, publication date, definition and treatment of each important input. It should also record material changes and the reason for making them. In this project, the LSEG source register distinguishes the valuation reference date from the later publication of the buyback proxy.

Checking should be sufficiently independent to challenge the result. Repeating the same formula in two cells proves little if both contain the same mistake. Comparing the workbook with a separately calculated result is more useful. The final review of LSEG checked all 50 sensitivity outputs independently, rather than relying only on the workbook's own status label.

8.3 Data protection and fairness

Professional use also raises questions beyond the model itself. Confidential information should be handled through approved systems. A process affecting customers needs a way to investigate errors and explain decisions. Bias can arise from the data selected or from how a result is used, even if the underlying calculation works as designed.

These cases use public company information and do not test customer decisions. Their numerical checks should therefore not be described as evidence that an AI system is fair, secure or suitable for every financial application.

8.4 UK and European context

The FCA's discussion of AI places its use within the responsibilities of financial firms and the risks to consumers and markets. The EU AI Act uses a framework based on the risk of particular

applications. A firm needs to assess the rules relevant to its own use, rather than assume that every AI tool has the same obligations (FCA, 2026; European Commission, 2026).

The practical point for this paper is that adopting a tool does not remove the need to decide who is responsible for it. Oversight needs to reflect what the system can influence and what would happen if it failed.

9 What the Python analysis establishes

9.1 From data to a result

The cases use Python for different analytical tasks. In the Tesco and Carillion screen, pandas converts annual financial data into ratios and flags, and matplotlib makes unusual cash-conversion results visible. In LSEG, a reusable DCF function converts assumptions into forecast cash flows and value. The workbook builder then records formulas, inputs and results in Excel.

Case Study 2B uses the model outputs and a separate Python calculation to examine governance. It distinguishes a repeatable result from an independently checked calculation, and both from a well-supported financial assumption. The enterprise-to-equity reconciliation is one internal check within that review.

9.2 Reproducibility and checking

The appendix shows the code that performs the key calculations alongside the outputs it produces. The accompanying Python files contain the complete LSEG builder, a runnable Tesco screen, the unchanged Carillion application of that screen and a script that exports the LSEG case-study results. This lets a reader follow an input through the calculation and repeat the analysis.

Reproduction and independent validation serve different purposes. Running the same function again establishes that its outputs can be repeated. Recalculating the same financial logic separately can reveal an implementation error. Neither check establishes that a forecast assumption is economically correct.

9.3 What remains unresolved

The Tesco and Carillion results depend on the selected accounting variables and reporting versions in their datasets. LSEG depends heavily on terminal value and on an incomplete ownership bridge. These limitations remain after the code runs successfully. Python makes their effect easier to examine, but financial interpretation is still needed.

10 Conclusion

The Tesco and Carillion comparison shows that a simple screen can miss a particular accounting issue while producing useful prompts elsewhere. Both the warning and the quiet result require interpretation.

The LSEG valuation is reproducible but depends heavily on terminal assumptions and incomplete ownership adjustments. Its governance review shows what the numerical checks establish and which questions remain unresolved.

Across the cases, reliable analysis requires tested calculations, clearly defined inputs and conclusions proportionate to the evidence. Automation makes the work easier to examine, while financial understanding remains essential.

11 Case study appendix

The three case studies follow the same structure: research question, evidence, method, Python implementation, results, limitations and checks, interpretation, and sources. Case Study 1 applies one screen to Tesco and Carillion. Cases 2A and 2B share the LSEG model.

Case Study 1

Tesco and Carillion financial reporting risk

Research question

Could a simple automated screen of annual financial statements identify unusual patterns that deserved further investigation? The same fixed Python rules were applied to Tesco from 2010 to 2015 and Carillion from 2010 to 2016. The cases examine earnings quality, working capital and the limits of a small retrospective screen.

Tesco provides an example where the screen did not identify a clear warning specific to the later commercial-income issue. Carillion tests whether the unchanged rules produced relevant prompts before its July 2017 profit warning. Neither result is treated as a prediction of misconduct or insolvency.

Evidence

The Tesco data come from the project workbook *Tesco_Master_Dataset_2010_2015.xlsx*. It contains revenue, profit before and after tax, operating cash flow, receivables, inventory and total assets, in £ millions. The analysis uses the collected series as an exploratory dataset. Its reporting versions have not all been reconciled into a consistent record of information available on each original publication date.

Tesco corrected its August 2014 profit expectation on 22 September 2014. The October estimate of the commercial-income overstatement was £263 million. Tesco later explained that this included £118 million affecting the half-year expectation, £70 million from the previous year and £75 million from earlier years. Its 2015 report subsequently increased the adjustment relating to prior years to £208 million (Tesco, 2015, p. 33; FCA, 2017).

The annual period ending in February 2014 preceded the August interim period and public disclosure. It would nevertheless be wrong to conclude that earlier years were unaffected. Timing matters, but it does not by itself explain why the screen failed.

Carillion data are transcribed from its 2010, 2012, 2013, 2015 and 2016 annual reports. The 2016 accounts are the last full annual report before the July 2017 profit warning. Carillion entered compulsory liquidation in January 2018 after an £845 million contract write-down. The historical outcome supplies context; it was not used to change the screening rules.

Method

The calculations used fixed rules in Python. This was an automated accounting screen, rather than a trained machine-learning model. It provides a transparent starting point for considering where more advanced AI analysis might help.

For each year, profit after tax less operating cash flow was divided by average total assets. This gives a broad accrual ratio. Average assets were calculated from the current and previous year-end balances. The first year therefore has no ratio. Profit before tax was not used in this calculation.

Receivables and inventory each triggered a warning if their annual growth exceeded revenue growth by more than five percentage points. This is a fixed screening assumption, not a threshold calibrated to either sector. A separate test compared the current accrual ratio with the median of the

preceding three annual ratios. Where the history was insufficient, the result was recorded as unavailable.

The rules can be reproduced by calculating each annual growth rate as the current balance divided by the previous balance, less one. The accrual test uses only earlier ratios in its comparison. Missing history must not be treated as a passed test.

Carillion uses the same calculations and warning rules as Tesco. Its reporting versions are not fully harmonised: 2012 and 2014 use later comparatives, while 2016 uses the original published accounts. The retained 2015 balances differ slightly from the restated comparatives in the 2016 report. The source record in the script documents these choices.

Python implementation

Python was used to load the annual data, calculate accounting ratios, apply the same rules to each year and export the results. pandas handles the tabular calculations and matplotlib draws the charts. The listings below set out the calculation steps. The accompanying tesco_analysis.py combines these rules with data loading, checks and output commands.

The input columns are renamed to Revenue, PBT, CFO, Receivables, Inventory, PAT and Total_Assets. The rows are sorted by Year. Missing values, duplicate years and gaps in the annual sequence stop the script before growth rates are calculated. Amounts remain in £ millions.

Listing 1 Accruals and annual growth

```
df["Total_Accruals"] = df["PAT"] - df["CFO"]
df["Average_Total_Assets"] = (
    df["Total_Assets"] + df["Total_Assets"].shift(1)
) / 2
df["Accruals_to_Assets"] = (
    df["Total_Accruals"] / df["Average_Total_Assets"]
)
for column in ["Revenue", "Receivables", "Inventory", "CFO", "PAT"]:
    df[column + "_YoY_%"] = df[column].pct_change(fill_method=None)
```

shift(1) supplies the previous year without moving later information into an earlier observation. pct_change returns fractional growth rates: 0.05 means 5% growth. The first year has no previous asset balance and therefore no accrual ratio.

Listing 2 Working capital warnings

```
df["Flag_Receivables_Growing_Faster"] = (
    df["Receivables_YoY_%"] > df["Revenue_YoY_%"] + 0.05
).astype("boolean")
df["Flag_Inventory_Growing_Faster"] = (
    df["Inventory_YoY_%"] > df["Revenue_YoY_%"] + 0.05
).astype("boolean")
for column, flag in [
    ("Receivables", "Flag_Receivables_Growing_Faster"),
    ("Inventory", "Flag_Inventory_Growing_Faster"),
]:
    valid = df["Revenue_YoY_%"].notna() & df[column + "_YoY_%"].notna()
    df.loc[~valid, flag] = pd.NA
```

The nullable boolean type allows the first year to remain unavailable. Without this step, a comparison involving a missing growth rate could appear as False and be mistaken for a clean result.

Listing 3 Accrual history and cash conversion

```
prior_median = df["Accruals_to_Assets"].shift(1).rolling(3).median()
valid = prior_median.notna()
df["Flag_Positive_Accrual_Build"] = pd.Series(
    pd.NA, index=df.index, dtype="boolean"
)
df.loc[valid, "Flag_Positive_Accrual_Build"] = (
    df.loc[valid, "Accruals_to_Assets"] > prior_median.loc[valid]
)
df["Cash_Conversion"] = df["CFO"] / df["PAT"].where(df["PAT"] != 0)
```

The three-year median uses prior observations only. Cash conversion is left unavailable when PAT is zero. A small or negative profit still produces a number, so those observations are retained and explained rather than treated as normal operating performance.

Listing 4 Separate deterioration screen introduced after review

```
df["Flag_Statutory_Loss"] = (df["PBT"] < 0) | (df["PAT"] < 0)
df["Flag_CFO_Collapse"] = df["CFO_YoY_%"] <= -0.30
df["Assets_YoY_%"] = df["Total_Assets"].pct_change(fill_method=None)
df["Flag_Asset_Contraction"] = df["Assets_YoY_%"] <= -0.10
df["Flag_Revenue_And_Profit_Deterioration"] = (
    (df["Revenue_YoY_%"] < 0) & (df["PAT_YoY_%"] < 0)
)
```

This later screen is exported separately in tesco_deterioration_review.csv. The original rules are retained in tesco_redflags_updated.csv. Keeping the two outputs separate prevents the later rule changes from being counted as successful advance detection.

carillion_analysis.py applies the original accounting screen without changing its five-percentage-point working-capital threshold or three-year accrual-history rule. Its 2010 to 2016 data are embedded and its output is retained separately from the Tesco results.

Results

Table 1 Results of the original Tesco screen

Year	Revenue growth	Receivables warning	Inventory warning	Accrual ratio
2010	Unavailable	Unavailable	Unavailable	Unavailable
2011	7.07%	Yes	Yes	-3.36%
2012	4.90%	Yes	Yes	-3.25%
2013	1.42%	No	No	-5.38%
2014	-1.96%	No	No	-4.42%
2015	-2.00%	No	No	-13.24%

Source: recalculation from the project dataset. The accrual ratio is profit after tax less operating cash flow, divided by average assets. The separate accrual-history warning was unavailable until 2014 and did not trigger in either 2014 or 2015.

The working-capital rules flagged 2011 and 2012. These earlier movements could have several explanations, including expansion, acquisitions or changes in the business mix. The test does not establish that they were evidence of misstatement. The absence of later warnings also does not show that the accounts were free from problems.

Cash conversion provided another useful lesson. Dividing operating cash flow by the £120 million profit-after-tax input in 2013 produces a ratio of about 23.6 times. In 2015 the profit input is negative. These ratios are arithmetically valid but are poor comparisons with normal trading years.

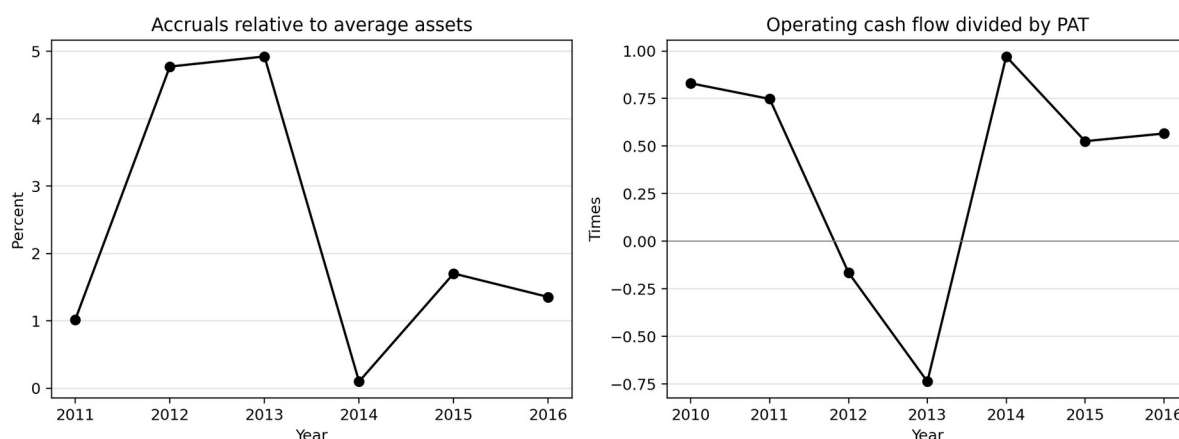
After reviewing the original result, a separate deterioration test examined a statutory loss, a fall in operating cash flow of at least 30%, a fall in assets of at least 10%, and simultaneous declines in revenue and profit. All four conditions were present in the 2015 dataset. This describes financial deterioration after the disclosure. It is not evidence of advance detection.

Table 2 Results of the unchanged Carillion screen

Year	Revenue growth	Receivables warning	Inventory warning	Accrual ratio	Cash conversion
2010	Unavailable	Unavailable	Unavailable	Unavailable	0.83 times
2011	-1.97%	Yes	Yes	1.02%	0.75 times
2012	-11.73%	Yes	No	4.77%	-0.17 times
2013	-9.10%	Yes	No	4.92%	-0.74 times
2014	4.84%	No	No	0.10%	0.97 times
2015	13.07%	No	Yes	1.70%	0.53 times
2016	11.24%	Yes	Yes	1.35%	0.57 times

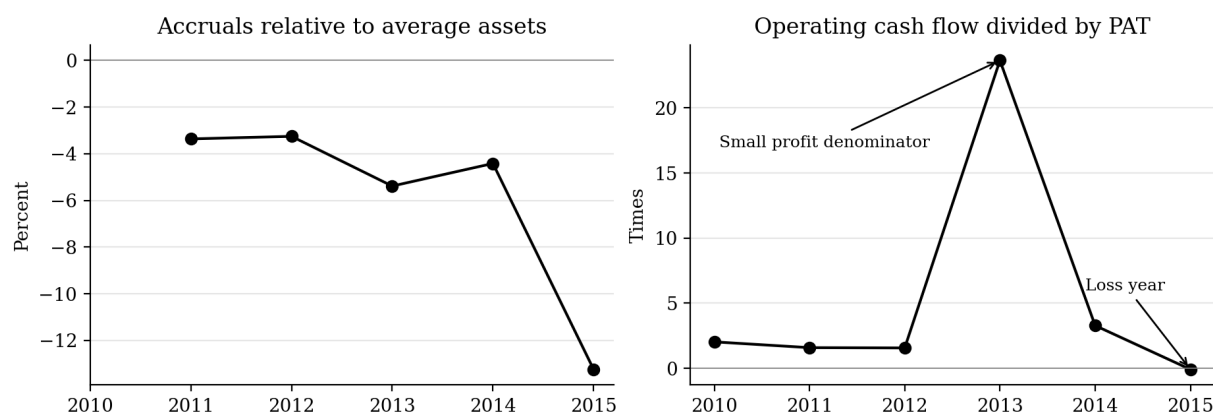
Source: carillion_redflags_updated.csv. The screen raised recurring receivables warnings from 2011 to 2013. In 2012 and 2013, operating cash flow was negative despite reported profit after tax. In 2016, both working-capital warnings triggered: receivables rose 31.0% and inventory 22.6%, compared with revenue growth of 11.2%.

Figure 1 Charts generated by the Carillion Python script



Source: carillion_analysis.py. Accrual ratios were high in 2012 and 2013, and cash conversion was weak in 2012, 2013, 2015 and 2016. The 2016 accounts preceded the July 2017 profit warning. The charts were produced retrospectively and do not demonstrate advance prediction.

Figure 2 Charts generated by the Tesco Python script



Source: tesco_analysis.py using the supplied annual dataset. The missing 2010 accrual point reflects the absence of 2009 assets. Cash conversion reaches 23.64 times in 2013 because PAT is small, and becomes negative in the 2015 loss year. The chart preserves these results so their limitations remain visible.

Table 3 Python output from the later deterioration screen

Year	Statutory loss	CFO fall of at least 30%	Asset fall of at least 10%	Revenue and profit fall
2014	No	No	No	No
2015	Yes	Yes	Yes	Yes

Source: tesco_deterioration_review.csv. These rules were introduced after the original results were reviewed and identify the visible 2015 deterioration.

Limitations and checks

The working-capital and accrual results were recalculated from the workbook. This confirms their arithmetic for the supplied inputs. It does not resolve every difference between original and later comparative financial statements. Changes in reporting scope, discontinued operations and exceptional items can affect comparisons.

The Tesco dataset has only six annual observations. It has no supplier contracts, internal commercial-income schedules or detailed evidence about when performance conditions were met. Receivables and inventory growth are indirect indicators of a problem involving supplier income and cost accruals. Without a labelled peer sample, false-positive rates and detection accuracy cannot be estimated reliably.

The later deterioration rules answer a different question from the original screen and were specified after its results were reviewed. Their success in identifying the 2015 loss must not be used to improve the claimed performance of the earlier test.

Carillion has one known outcome, seven annual observations and no healthy sector comparison. The screen omits debt, pensions, contract provisions, reverse factoring and detailed project data. Construction and retail also have different working-capital patterns, so applying the same rule tests consistency rather than equal diagnostic value. These limits prevent an estimate of detection accuracy.

Interpretation

The two companies show both sides of the screen. It did not isolate Tesco's commercial-income issue, but produced recurring working-capital warnings and weak cash-conversion readings for Carillion. The latter are ratio observations, not an additional coded warning rule. The comparison supports further investigation rather than an automatic conclusion.

The useful finding is that a consistent calculation can still be an unsuitable test for the question being asked. More data would help only if it contained information relevant to the accounting treatment. A further study could use interim disclosures and supplier-income notes, while separating documents published before and after the September announcement.

For audit work, the result would lead to questions about contracts and recognition policies. For an investor or transaction adviser, it would raise questions about sustainable earnings and cash generation before relying on a valuation. The lesson is to investigate the reason for a signal and to examine the limitations of a quiet result with equal care.

Sources

Tesco PLC (2014). Annual Report and Financial Statements 2014. Group financial statements and five-year record. [Source](#)

Tesco PLC (2015). Annual Report and Financial Statements 2015. Commercial income, p. 33, and independent auditor's report. [Source](#)

Financial Conduct Authority (2017). Tesco to pay redress for market abuse. 28 March, subsequently updated. [Source](#)

Carillion plc (2016). Annual Report and Accounts 2016. Group financial statements, pp. 90, 93 and 94. [Source](#)

House of Commons Work and Pensions and Business, Energy and Industrial Strategy Committees (2018). Carillion. Profit warning, contract write-down and financial failure. [Source](#)

Project evidence: `Tesco_Master_Dataset_2010_2015.xlsx`, `tesco_analysis.py` and `carillion_analysis.py`. The Carillion script embeds the documented annual-report data and exports its results separately. The two historical series have source-vintage limitations and are not presented as a fully harmonised backtest.

Case Study 2A

LSEG valuation and investment judgement

Research question

Can a Python valuation model produce a useful investment assessment of London Stock Exchange Group? This case considers the link between financial evidence, valuation assumptions and an investment conclusion. The workbook combines a five-year discounted cash flow model, a comparable-company screen, three operating scenarios and two sensitivity tables.

The financial base is FY2025. The market reference date is 26 February 2026. This is a retrospective valuation exercise, not a test of an investment decision made using only information published by that date.

Evidence

The main operating inputs are income excluding recoveries of £8,986 million, adjusted EBITDA of £4,523 million and adjusted operating profit of £3,506 million. The model uses income excluding recoveries as its revenue measure. It should not be confused with statutory revenue (LSEG, 2026a).

The £81.266 reference price is the average price paid in LSEG's own purchase programme on 26 February. The 27 February announcement also gives the prospective post-cancellation voting-share figure of 505.332519 million. These are explicitly labelled proxies. The price is not the official market close, and the share count is not a diluted weighted-average denominator (LSEG, 2026c).

Python generated LSEG_Corporate_Valuation_2025_Python_Only.xlsx using build_lseg_valuation_python_only.py. The source register distinguishes reported figures from assumptions and identifies incomplete peer evidence. The recorded calculation review is dated 9 September 2026 and is examined in Case Study 2B.

Method

Income growth and EBITDA margins determine forecast operating profit. Tax, depreciation, capex and working-capital investment convert this into unlevered free cash flow, before financing payments. The weighted average cost of capital, or WACC, discounts each annual cash flow.

WACC is approximately 7.69%, using assumed market rates, beta, debt cost and capital weights. These inputs are modelling choices. Operating net debt is used as a simplified financing weight.

The base case uses annual income growth of 7.0%, 6.5%, 6.0%, 6.0% and 5.5%, with the EBITDA margin rising to 54% in 2030. Capital expenditure falls from 9.5% of income in 2026 to 8% in 2029 and remains there in 2030. The tax assumption is 24.5%, with annual working-capital investment of 1.5% of income.

A terminal value estimates cash flows after 2030 using 2.5% perpetual growth. Operating net debt of £8,175 million is deducted from enterprise value to obtain a simplified equity value, which is divided by the voting-share proxy. ICE, CME and Nasdaq provide an indicative comparison using enterprise value divided by adjusted operating profit. A 70% DCF and 30% comparable-company weighting provides an additional illustration.

Python implementation

The supplied file build_lseg_valuation_python_only.py contains the complete workbook builder. Its dcf_case function calculates each forecast year in Python before XlsxWriter writes the formulas and their saved results to Excel. The following excerpts come from that builder; line breaks have been adjusted for the page.

The constants are a 4.5% risk-free rate, 4.5% equity risk premium, beta of 0.90, 4.5% debt cost and tax rate of 24.5%. The price, voting shares and operating net debt use the proxies described above.

Listing 5 WACC and capital expenditure assumptions

```
MARKET_CAP = REFERENCE_PRICE * SHARE_COUNT
DEBT_WEIGHT = NET_DEBT / (NET_DEBT + MARKET_CAP)
EQUITY_WEIGHT = MARKET_CAP / (NET_DEBT + MARKET_CAP)
COST_OF_EQUITY = RISK_FREE_RATE + LEVERED_BETA * EQUITY_RISK_PREMIUM
AFTER_TAX_COST_OF_DEBT = PRE_TAX_COST_OF_DEBT * (1 - TAX_RATE)
WACC = (EQUITY_WEIGHT * COST_OF_EQUITY
        + DEBT_WEIGHT * AFTER_TAX_COST_OF_DEBT)
CAPEX_RATES = [0.095, 0.090, 0.085, 0.080, 0.080]
```

The model raises income by each annual growth assumption and moves the EBITDA margin in equal steps towards the terminal-year target. Depreciation is 11.3% of income. Working-capital investment is modelled as an annual cash outflow of 1.5% of income, rather than 1.5% of the change in income.

Listing 6 The DCF function from the supplied builder

```
def dcf_case(growth, target_margin, terminal_growth, wacc=WACC):
    revenue, margin = 8986.0, 4523.0 / 8986.0
    rows = []
    for year, g in enumerate(growth, 1):
        revenue *= 1 + g
        margin += (target_margin - 4523.0 / 8986.0) / 5
        ebitda = revenue * margin
        da = revenue * 0.113
        ebit = ebitda - da
        tax = ebit * TAX_RATE
        capex = revenue * CAPEX_RATES[year - 1]
        nwc = revenue * 0.015
        fcf = ebit - tax + da - capex - nwc
        discount = 1 / (1 + wacc) ** year
        rows.append((revenue, margin, ebitda, da, ebit, tax,
                    capex, nwc, fcf, discount, fcf * discount))
    pv_forecast = sum(x[10] for x in rows)
    terminal = rows[-1][8] * (1 + terminal_growth) / (wacc - terminal_growth)
    enterprise = pv_forecast + terminal * rows[-1][9]
    equity = enterprise - NET_DEBT
    return rows, enterprise, equity / SHARE_COUNT
```

Each row stores income, margin, EBITDA, depreciation, EBIT, tax, capex, working-capital investment, free cash flow, the discount factor and discounted free cash flow. The final three steps add terminal value, deduct operating net debt and divide by the voting-share proxy. WACC must exceed terminal growth for this perpetuity calculation.

The companion file `lseg_case_outputs.py` imports this function and reads the three scenarios from the workbook. The excerpt below exports both sensitivity grids. Every operating-grid cell calls `dcf_case` again, so all five forecast cash flows change with the growth and margin assumptions.

Listing 7 Recalculating both sensitivity grids

```
discount_grid = [
    [dcf_case(base_growth, base_margin, g, wacc=w)[2]
     for g in terminal_growth_rates]
    for w in waccs
]
operating_grid = [
    [dcf_case([growth] * 5, margin, base_terminal_growth)[2]
     for margin in margins]
    for growth in growth_rates
]
```

The arrays contain WACC values from 6.5% to 8.5%, terminal growth from 1.5% to 3.5%, annual income growth from 4% to 8% and terminal-year margins from 50% to 58%. The two grids answer different questions. The first changes discounting and terminal growth while holding the base operating forecast fixed. The second changes the operating forecast while holding WACC and terminal growth fixed.

Listing 8 The comparable company calculation

```
peer_multiples = []
for row in (8, 9, 10):
    price, shares = peers.cell(row, 3).value, peers.cell(row, 4).value
    debt, profit = peers.cell(row, 6).value, peers.cell(row, 8).value
    peer_multiples.append((price * shares + debt) / profit)
peer_median = median(peer_multiples)
comps_value = (peer_median * 3506 - NET_DEBT) / SHARE_COUNT
blended_value = 0.70 * base_value + 0.30 * comps_value
```

The peer calculation uses each company's price, share count, net debt and adjusted operating profit from the Comps worksheet. Market capitalisation and net debt must use the same currency as operating profit. The resulting multiple can then be applied to LSEG's £3,506 million adjusted operating profit. The median and blend reproduce the existing model; they do not resolve the provisional peer inputs.

Results

Table 4 Conditional LSEG valuation results

Method or scenario	Value per share
Downside DCF	£95.41
Base DCF	£122.92
Upside DCF	£150.91
Comparable-company indication	£123.96
70% DCF and 30% comparable blend	£123.23

Source: LSEG project workbook, DCF, Comps and Scenarios worksheets. The base DCF is approximately 51.3% above the programme-price proxy. This is a difference between a model value and a historical price reference, rather than a forecast return.

The base-case enterprise value is approximately £70.29 billion. About 80.1% comes from discounted terminal value. Reducing only the 2030 EBITDA margin from 54% to 52.7% reduces the indicated value to £119.21 per share. This shows why the assumptions beyond the immediate forecast period deserve close attention.

Table 5 Base forecast calculated in Python in £ millions

Year	Income	EBITDA	Capex	Free cash flow	Discounted cash flow
2026	9,615	4,910	913	2,916	2,707
2027	10,240	5,304	922	3,213	2,770
2028	10,854	5,702	923	3,520	2,818
2029	11,506	6,129	920	3,853	2,864
2030	12,138	6,555	971	4,132	2,852

Source: lseg_base_forecast.csv. The discounted forecast cash flows total £14,012 million. Discounted terminal value adds £56,278 million, producing the £70,290 million enterprise value. This bridge explains the 80.1% dependence on terminal value.

Table 6 WACC and terminal growth sensitivity in £ per share

WACC	1.5%	2.0%	2.5%	3.0%	3.5%
6.5%	133.65	147.78	165.43	188.13	218.39
7.0%	119.68	131.02	144.89	162.21	184.49
7.5%	108.05	117.32	128.46	142.06	159.07
8.0%	98.21	105.91	115.02	125.95	139.30
8.5%	89.78	96.26	103.83	112.77	123.49

Columns are terminal growth rates. The base operating forecast is unchanged. The grid does not include the exact 7.69% base WACC, so no cell is expected to equal £122.92.

Table 7 Annual income growth and final margin sensitivity in £ per share

Growth	50%	52%	54%	56%	58%
4%	99.68	104.84	110.00	115.15	120.31
5%	104.86	110.26	115.66	121.07	126.47
6%	110.23	115.89	121.54	127.20	132.85
7%	115.81	121.72	127.64	133.56	139.47
8%	121.58	127.77	133.96	140.15	146.33

Columns are 2030 EBITDA margins. Growth is constant across all five forecast years in each row. WACC remains approximately 7.69% and terminal growth is 2.5%. The 6% growth and 54% margin cell is £121.54, rather than the base DCF of £122.92, because the base case uses a declining sequence of annual growth rates. Both tables reproduce all 50 outputs from the existing workbook.

Table 8 Comparable company multiples reproduced in Python

Company	EV to adjusted operating profit
ICE	18.87 times
CME	24.98 times
Nasdaq	20.20 times
Median	20.20 times

Source: lseg_peer_multiples.csv, using the Comps worksheet inputs. Applying the unrounded median produces £123.96 per LSEG share. The 70% DCF and 30% comparable-company blend is £123.23.

Limitations and checks

The valuation was reviewed and corrected before the results reported here were retained. Case Study 2B sets out the calculation checks, corrected issues and outstanding governance questions.

The recorded tests cover spreadsheet recalculation, independent scenario and sensitivity calculations, and changes to selected inputs. They support calculation consistency for the tested inputs. They do not establish that the operating forecasts will be achieved.

The 54% margin is above an illustrative guidance-based path reaching about 52.7% in 2029. It assumes further improvement in 2030. Management's guidance also uses a constant-currency basis, so applying it to reported sterling income without a separate currency bridge is an approximation. Some peer capital-structure and P/E inputs remain approximate, and the enterprise-to-equity calculation does not fully reconcile minority interests, leases or subsequent capital changes.

Interpretation

The model supports further investigation of the investment case. The apparent upside alone does not justify a purchase: terminal value, profitability assumptions and the ownership bridge could materially change the result.

The workbook makes the calculation reproducible. Judgement is still needed about whether the assumptions and inputs represent the business well enough to support an investment decision.

Sources

London Stock Exchange Group (2026a). Preliminary results for the year ended 31 December 2025. 26 February. Financial highlights, cash flow, operating net debt and alternative performance measures. [Source](#)

London Stock Exchange Group (2026b). Preliminary results announcement. 2026 guidance and medium-term guidance for 2027 to 2029. [Source](#)

London Stock Exchange Group (2026c). Transactions in own securities. RNS, 27 February, reporting purchases on 26 February. [Source](#)

Project evidence: the complete `build_lseg_valuation_python_only.py`, its generated workbook and `lseg_case_outputs.py`. The latter exports the forecast, scenarios, comparable multiples and both sensitivity grids for this paper. All are included in `AI_in_Finance_Python_Work.zip`.

Case Study 2B

LSEG model governance

Research question

What evidence and controls are needed before relying on the LSEG valuation in Case Study 2A? This case reviews calculation accuracy, source quality, assumptions and the treatment of unresolved issues. It uses the same workbook so that the valuation and its governance can be examined together.

The review distinguishes completed numerical checks from controls that would be needed in professional use. It assesses a student valuation exercise; no formal institutional approval or complete model-governance process is claimed.

Evidence

The evidence consists of the LSEG workbook, its Python builder, the separate checking script and the saved review records. The source register records the financial inputs and assumptions. The market-price and voting-share proxies were published on 27 February 2026, after the 26 February reference date, making this a retrospective exercise.

The comparable-company screen uses ICE, CME and Nasdaq. Their different business mixes, accounting adjustments and approximate capital-structure inputs limit comparability. These are evidence-quality issues that remain even when the multiples have been calculated correctly.

Method

The review separates three questions: does the calculation work, are the inputs supported, and is the result suitable for its intended use? Recalculation and separate Python checks address the first question. Source review and sensitivity analysis help address the other two, but do not remove the need for judgement.

Calculation checks compare the three operating scenarios and all 50 sensitivity cells with a separate implementation. The earlier review also recalculated the workbook in a separate spreadsheet engine and changed the selected scenario and a final-year growth input to check that linked results responded.

Source and assumption review considers publication dates, input definitions, peer comparability and the large contribution from terminal value. A professional process would also assign responsibility for updates, retain a change record and require review before revised outputs were used. These are proposed controls, not additional tests claimed as completed.

Python implementation

`validate_lseg.py` recalculates the DCF from workbook inputs without using the builder's `dcf_case` function for the checking calculation. It compares the results with the model and writes `lseg_independent_review.json`. The excerpt below shows the scenario comparison after the independent function has calculated each result.

Listing 9 Comparing scenarios with a separate calculation

```
results = {k: independent(*args) for k, args in cases.items()}
mod = runpy.run_path(str(ROOT / "build_lseg_valuation_python_only.py"))
python_diffs = {
    k: mod["dcf_case"](*args)[2] - results[k]["value"]
    for k, args in cases.items()
}
```

Listing 10 The enterprise to equity bridge in Python

```
results = {name: dcf_case(*args) for name, args in cases.items()}
bridge = []
for name, (forecast_rows, enterprise, per_share) in results.items():
    equity = enterprise - NET_DEBT
    if not math.isclose(equity / SHARE_COUNT, per_share):
        raise AssertionError("Enterprise to equity bridge does not reconcile")
    bridge.append({
        "Scenario": name, "Enterprise_value": enterprise,
        "Net_debt": NET_DEBT, "Equity_value": equity,
        "Shares_millions": SHARE_COUNT, "Value_per_share": per_share,
    })
```

The existing lseg_transaction_bridge.csv records this internal reconciliation across scenarios. Its filename is retained so the evidence remains traceable. The check confirms that enterprise value less operating net debt, divided by shares, agrees with the shared DCF result; it does not establish that every ownership adjustment has been included.

The separate calculation in Listing 9 can expose implementation differences. The bridge in Listing 10 checks internal consistency. Neither verifies the economic assumptions. The full checking script also compares all 50 sensitivity cells and retains the numerical differences in its output record.

Results

Table 9 Base valuation used in the governance review

Measure	Model result
Enterprise value	£70,290 million
Operating net debt deducted	£8,175 million
Simplified equity value	£62,115 million
Voting-share proxy	505.333 million
Indicated value per share	£122.92

Source: LSEG project workbook, DCF worksheet. Amounts are rounded. The simplified bridge deducts operating net debt only; the completeness of the ownership adjustments remains an open review point.

The separate calculation matched three scenarios and 50 sensitivity cells to floating-point precision, with differences below £0.000001 per share. The earlier spreadsheet-engine review compared 277 formula results and recorded no formula errors or material changes to saved values.

The recorded scenario-switch and final-year growth checks updated the linked valuation and check outputs. Earlier review had corrected a discount-rate mismatch, operating sensitivity logic, the scenario check and a comparable P/E median. The P/E median does not drive the reported valuation.

Table 10 Python enterprise to equity reconciliation across scenarios

Scenario	Enterprise value £m	Net debt £m	Equity value £m	Value per share
Downside	56,389	8,175	48,214	£95.41
Base	70,290	8,175	62,115	£122.92
Upside	84,433	8,175	76,258	£150.91

Source: lseg_transaction_bridge.csv. Net debt and shares are held constant across the three scenarios. This table demonstrates the internal reconciliation, while the separate Python and spreadsheet checks provide additional calculation evidence.

Limitations and checks

Passing these checks does not mean the model is ready for an investment decision. About 80.1% of base enterprise value comes from terminal value. Reducing the final margin from 54% to 52.7% lowers the DCF from £122.92 to £119.21 per share. The WACC and terminal-growth grid spans £89.78 to £218.39, showing the importance of assumption review.

The operating net-debt measure excludes lease liabilities and incorporates regulatory and operational cash adjustments. Minority interests, dilution and subsequent capital changes are not fully reconciled. Some peer inputs remain approximate. These unresolved items should be recorded alongside the valuation rather than hidden behind a passing calculation status.

The portable Python check reproduces the independent scenario and sensitivity calculations. The 277-cell recalculation and input-change tests are retained historical review records; running that script alone does not repeat them. No forecasting backtest, external model approval or test of confidential-data handling is included.

Interpretation

The model is suitable for examining valuation mechanics and testing assumptions within the stated scope. Before relying on its investment conclusion, the outstanding source and ownership questions would need to be resolved and the calculations checked again after any changes.

This case shows that governance involves more than a model reporting that it has passed. It requires knowing what was checked, retaining the evidence and making unresolved questions visible. Case Study 2A supplies the valuation; Case Study 2B explains the basis and limits of confidence in that valuation.

Sources

London Stock Exchange Group (2026a). Preliminary results for the year ended 31 December 2025. 26 February. Financial highlights, cash flow, operating net debt and alternative performance measures. [Source](#)

London Stock Exchange Group (2026c). Transactions in own securities. RNS, 27 February, reporting purchases on 26 February. [Source](#)

Intercontinental Exchange (2026). Strong full year 2025 results. 5 February. Adjusted operating-income reconciliation. [Source](#)

CME Group (2026). Full year 2025 results. 4 February. Adjusted operating-income reconciliation. [Source](#)

Nasdaq (2026). Fourth quarter and full year 2025 results. 29 January. Non-GAAP operating-income reconciliation. [Source](#)

Project evidence: the LSEG builder and workbook; validate_lseg.py; lseg_independent_review.json; lseg_original_review_record.json; lseg_original_scenario_switch_record.json; and the enterprise-to-equity reconciliation in lseg_transaction_bridge.csv. These files are included in AI_in_Finance_Python_Work.zip.

Reproducing the Python calculations

AI_in_Finance_Python_Work.zip contains the source files, input workbooks and generated outputs. Extract the archive, open its Python_work folder and run the commands below. The scripts use paths relative to their own location. Python 3, pandas, openpyxl, matplotlib and XlsxWriter are required; the tested versions are recorded in requirements.txt.

Commands to reproduce the analysis

```
python tesco_analysis.py
python carillion_analysis.py
python lseg_case_outputs.py
```

The first two commands regenerate the accounting results and charts. The third exports the LSEG outputs and compares all 50 sensitivity values with the saved workbook. Run python validate_lseg.py for the separate scenario and sensitivity calculation. The earlier spreadsheet-engine and input-change tests are retained as review records; this command does not repeat them.

To rebuild the valuation workbook beside its source file, run the following command. The builder includes the source register and model checks.

Command to rebuild the valuation workbook

```
python build_lseg_valuation_python_only.py
```

Table 11 Python files and generated evidence

File	Purpose
tesco_analysis.py	Loads Tesco data, calculates ratios and flags, and draws charts
carillion_analysis.py	Applies the unchanged Tesco screen to Carillion data through 2016
build_lseg_valuation_python_only.py	Complete supplied LSEG workbook builder and DCF function
lseg_case_outputs.py	Exports the LSEG forecasts, sensitivities, peers and equity bridge
validate_lseg.py	Checks the scenarios and sensitivity cells using separate calculations
results/tesco_redflags_updated.csv	Original accounting-screen output
results/tesco_deterioration_review.csv	Later review output kept separate
results/carillion_redflags_updated.csv	Comparative accounting-screen output through 2016
results/lseg_base_forecast.csv	Intermediate forecast cash flows
results/lseg_wacc_sensitivity.csv	All 25 discount-rate sensitivity results
results/lseg_operating_sensitivity.csv	All 25 operating sensitivity results
results/lseg_transaction_bridge.csv	Scenario enterprise and equity values

The README explains each file and the distinction between reproduced outputs, separate calculations and retained review records.

12 References

References for the discussion and practical case studies are listed below. Further input definitions and source locations are recorded in the project workbooks.

Bank of England and Financial Conduct Authority (2024). Artificial intelligence in UK financial services. Survey published 21 November. [Source](#)

CME Group (2026). Full year 2025 results. 4 February. Adjusted operating-income reconciliation. [Source](#)

European Commission (2026). AI Act. Accessed 9 September 2026. [Source](#)

Financial Conduct Authority (2017). Tesco to pay redress for market abuse. 28 March, subsequently updated. [Source](#)

Financial Conduct Authority (2026). AI in financial services. Accessed 9 September 2026. [Source](#)

Intercontinental Exchange (2026). Strong full year 2025 results. 5 February. Adjusted operating-income reconciliation. [Source](#)

London Stock Exchange Group (2026a). Preliminary results for the year ended 31 December 2025. 26 February. Financial highlights, cash flow, operating net debt and alternative performance measures. [Source](#)

London Stock Exchange Group (2026b). Preliminary results announcement. 2026 guidance and medium-term guidance for 2027 to 2029. [Source](#)

London Stock Exchange Group (2026c). Transactions in own securities. RNS, 27 February, reporting purchases on 26 February. [Source](#)

Nasdaq (2026). Fourth quarter and full year 2025 results. 29 January. Non-GAAP operating-income reconciliation. [Source](#)

Tesco PLC (2014). Annual Report and Financial Statements 2014. Group financial statements and five-year record. [Source](#)

Tesco PLC (2015). Annual Report and Financial Statements 2015. Commercial income, p. 33, and independent auditor's report. [Source](#)

Project files: Tesco_Master_Dataset_2010_2015.xlsx;

LSEG_Corporate_Valuation_2025_Python_Only.xlsx; build_lseg_valuation_python_only.py.

These contain the working data and calculations referred to in the appendix.

Carillion plc (2010). Annual Report and Accounts 2010. Group financial statements, pp. 54, 57 and 58. [Source](#)

Carillion plc (2012). Annual Report and Accounts 2012. Group financial statements, pp. 64, 67 and 68. [Source](#)

Carillion plc (2013). Annual Report and Accounts 2013. Group financial statements, pp. 74, 77 and 78. [Source](#)

Carillion plc (2015). Annual Report and Accounts 2015. Group financial statements, pp. 80, 83 and 84. [Source](#)

Carillion plc (2016). Annual Report and Accounts 2016. Group financial statements, pp. 90, 93 and 94. [Source](#)

House of Commons Work and Pensions and Business, Energy and Industrial Strategy Committees (2018). Carillion. Profit warning, contract write-down and financial failure. [Source](#)